Recapitulare EGC

Banda grafica fixa

Transformari (in ordine):

- Transformare varfuri (vertex shader in banda grafica programabila)
- Asamblare primitive
- Decupare, Rasterizare
- Eliminare parti nevizibile (fragment shader in banda grafica programabila)
- Frame Buffer

Transformari elementare

- translatie (modificarea pozitiei - factori de translatie)
- scalare fata de origine (marirea/micsorarea coordonatelor)
  - uniforma (sx = sy)
  - neuniforma
- roatatie fata de origine (unghiul in origine)
  - x' = x * cos(u) - y * sin(u)
  - y' = x * sin(u) + y * cos(u)
- oglindirea
- forfecarea

Coordonate omogene - reprezentare prin vectori coloana

Transformare compusa

Scalarea fata de un punct fix

- Translatia in origine T(-fx, -fy)
- Scalarea fata de origine S(sx, sy)
- Translatia inversa de la punctul 1: T(xf, xy)

> *Se executa invers T(xf, xy) S(sx, sy) T(-fx, -fy) * vector coloana*

Rotatia fata de un punct fix

- Translatia in origine T(-fx, -fy)
- Rotatia fata de origine R(u)
- Translatia inversa de la punctul 1: T(xf, xy)

> *Se executa invers T(xf, xy) R(u) T(-fx, -fy) * vector coloana*

Oglindirea fata de ox

- Scalare pe ox cu 1
- Scalare pe oy cu -1

Oglindirea fata de ox

- Scalare pe ox cu -1
- Scalare pe oy cu 1

Oglindirea fata o dreapta oarecare

- Translatie astfel incat dreapta d sa treaca prin origine T(-x1, -y1)
- Rotatie astfel incat dreapta d sa se suprapuna peste OX: R(-u)
- Oglindirea fata de OX: Ox
- Rotatia inversa de la punctul 2: R(u)
- Translatia inversa de la punctul 1: T(x1, y1)

> *Se executa invers: T(x1, y1) R(u) Ox R(-u) T(-x1, -y1) * vector coloana*

Forfecarea fata de ox

- fx factor de forfecare de-a lungul axei ox
- x' = x + y * fx
- y' = y

Forfecarea fata de oy

- x' = x
- y' = y + x * fx

Forfecarea fata de ambele axe

- x' = x + y * fx
- y' = y + x * fx

Transformari 2D ale sistemelor de coordonate

- Translatia
  - originea ajunge din O in O'
  - axele raman paralele cu precedente
  - x' = x - tx, y' = y - ty
- Scalare
  - x' = x/sx
  - y' = y/sy
- Rotatie
  - x' = x * cos(u) - y * sin(u)
  - y' = x * sin(u) + y * cos(u)

> *Transformare de instantiere: Sistem de coordonate obiect -> Sistem de coordonate global*

- Transformare de instantiere
  - 1) Scalare (sx, sy) => inmultirea
  - 2) Oglindire => -y
  - 3) Translatie (x1, y1) => adunarea
  - x' = x * sx + tx
  - y' = -y * sy + ty

- Transformare fereastra-poarta
  - Translatie(xnew, ynew)
  - Scalare(w/W, h/H)
  - Fereastra - zona dreptunghiulara cu laturile paralele cu axele sistemului de coordonate real ce incadreaza imaginea reprezentata in spatiul 2D real
  - Poarta - zona dreptunghiulara cu laturile paralele cu axele sistemului de coordonate dispozitiv (in care se va face afisarea)

- Centrarea in poarta de afisare - translatie suplimentara
  - tx' = (xp*maxim* - *xp*minim - sx * (xf*max* - *xf*min)) / 2
  - ty' = (yp*maxim* - *yp*minim - sy * (yf*max* - *yf*min)) / 2
- Afisare pe ecran
  - xp = xf * sx + tx
  - yp = yp*min* + *yp*max - (yf * sy + ty)

Sisteme de coordonate 3D

> *Sistemele se diferentiaza dupa sensul de crestere a axei OZ*

Sistem de coordonate dreapta (rhc)

Sistem de coordonate stanga (lhc)

Transformari geometrice 3D:

- Translatia (ca la 2D)
- Scalarea (ca la 2D)
- Rotatia:
  - in jurul unei axe a sistemului de coordonate (ca la 2D)
  - in jurul dreptei oarecare
    - translatie ca dreapta sa treaca prin origine
    - aliniere dreapta cu una din axe (rotatia alfa si beta pe celalalte doua axe)
    - rotatia propriu zisa
    - transformarea inversa de la aliniere

        - transformarea inversa a translatiei
- Oglindirea (ca o scalare, EX axa XOY: S(1, 1, -1))
- Forfecarea
    - Exemplu axa z: x' = x +f * z, y' = y + t * z; z' = z

Proiectii

> *Se caracterizeaza prin: centru de proiectie, proiector, plan de proiectie*

Clasificare in functie de pozitia observatorului:

- Proiectii plane
    - perspectiva glm::frustum
    - paralele glm::ortho
        - oblice (x' = x + z * f * cos(a); y' = y + z * f * sin(a), directia[x', y', -1])
            - cavalier (a = 45 grade)
            - cabinet (a = 63.4 grade)
        - ortogonale
            - axonometrice (pastreaza proportiile)
                - proiectia trimetrica
                    - fx != fy != fz
                - proiectia dimetrica
                    - fz oarecare, fx = fy
                    - fx oarecare, fz = fy
                    - fy oarecare, fz = fy
                - proiectia izometrica
                    - fx = fy= fz

        - vederi
            - principale
                - dreapta
                - stanga
                - fata
                - spate
                - sus
                - jos
            - auxiliare
            - sectiune
- Proiectie perspectiva: centrul de proiectie se afla la distanta finita fata de planul de proiectie
    - prezinta o imagine realista, nu ofera o reprezentare exacta a dimensiunilor si formelor obiectelor
    - efectul de micsorare - dimensiunea obiectului proiectat este invers proportionala cu distanta de la centrul de proiectie la obiect
    - nu se pastreaza intotdeauna paralelismul liniilor proiectate
    - liniile paralele care nu sunt paralele cu planul de proiectie se intersecteaza intr-un punct de convergenta
        - un punct de convergenta: o axa intersecteaza planul de proiectie
        - doua puncte de convergenta: doua axe intersecteaza planul de proiectie
        - trei puncte de convergenta: trei axe intersecteaza planul de proiectie
- Proiectie paralela: centrul de proiectie se afla la infinit fata de planul de proiectie
    - imagine mai putin realista
    - liniile se proiecteaza in linii paralele

Transformarea de vizualizare 3D

> *Sistem de coordonate obiect -> Sistem de coordonate real (global) -> sistem de vizualizare (definesc volum de vizualizare) -> Viewport 3D (decupare) -> 3D spre 2D (se lasa doar x si y, plan paralel cu xoy)*

Lantul de transformari OpenGL

- Transformare de modelare(obiect)-vizualizare(real)
  - trecere intre sistem de coordonate local in obiecte definite pe acelasi sistem de coordonate
- de proiectie (inclusiv decupare)
- impartire perspectiva
- transformarea in poarta de afisare

> *Transformare de modelare, vizualizare, proiectie sunt pastrate sub forma matricilor 4x4 in reprezentarea coloana*

- Sistem de vizualizare:
  - pozitie
  - directia de privire
  - orientarea: vector up
  - implicit
    - observator = origine
    - c: z negativ
    - up: y pozitiv

IMpartirea perspectiva

- Tranformarea din coordonate omogene in coordonate carteziene (w = 1)

- Volumul de vizualizare canonic devine: cub cu fetele paralele cu planele sistemului de coordonate:
  - -1 <= x <= 1
  - -1 <= y <= 1
  - -1 <= z <= 1

Transformarea in poarta de afisare

- Fata (-1, -1, -1) - (1, 1, -1) se mapeaza in poarta de afisare:
  - (x, y, z) => (xe, ye, z)
- Poarta de afisare:
  - glViewport(px, py, w, h) <- transformare fereastra-poarta

Banda grafica programabila

> *Lant de transformari: Sist obiect -> Sistem lumii reale -> Sistem de vizualizare (volum de vizualizare) -> Sistem de coordonate disp (din transformare de proiectie) -> viewport (proiectie)*

> *Varfuri in Vertex Shader -> Varfuri transformate in Asamblare primitive -> primitive asamblate in rasterizare si interpolare -> fragmente in fragment shader -> fragmente transformate in operatii raster -> actualizare frame buffer*

> *(x, y, z) -> Vertex Shader -> Asamblare primitive -> Decupare, Rasterizare -> Fragment Shader -> Frame Buffer*

Transmiterea parametrilor

> *Aplicatie (coordonate varfuri, normale, coordonate de texturare) -> Vertex Shader (x, y, z) -> Rasterizare -> Fragment Shader (adauga culoare) -> Frame Buffer*

Maparea texturilor 2D

> *Se defineste o mapare intre (x, y, z) din spatiul real in (s, t) in spatiul textura*

Pentru a determina culoarea din textura, se considera un punct (x, y, z) de pe suprafata, se mapeaza in spatiul textura, pe baza caruia se determina culoarea din textura: s = s(x, y, z); t = t(x, y, z); s, t apartin [0, 1]

Interpolare liniara pentru a mapa restul punctelor din spatiul real

Iluminare

Modele de culoare

- Modelul RGB (Red, Green, Blue)
- Modelul CMYK
  - datorita imperfectiunii cernelurilor se adauga K (blacK)
  - K = min(C, M, Y)
  - C = C - K
  - M = M - K
  - Y = Y - K
- Modelul HSV (Hue, Saturation, Value)

Modele pentru calculul luminii

- Modele empirice:
  - Lumina ambianta
  - Lumina difuza
  - Lumina speculara
- Modele analitice:
  - Modelul Lambert
  - Modelul Gouraud
  - Modelul Phong
- Iluminare locala
  - sursele de lumina
  - pozitia observatorului
  - proprietatile de material
- Iluminare globala
  - interactiunea dintre lumina provenita de la toate suprafetele scenei

Lumina ambientala

- Obiectele primesc lumina de la:
  - sursele de lumina
  - luminarea reflectata sau transmisa de celelalte obiecte ale scenei = lumina ambientala (este o sursa de lumina distribuita uniform in spatiu)
  - Iam = Ia * Ka
- Ia - sursa, intensitatea luminiii ambientale (este constanta pentru toate obiectele scenei)
- Ka - obiect/material, (0 <= Ka <= 1) este coeficitentul de difuzie a luminii ambientale

Lumina difuza

- se apropie cel mai mult la majoritatea luminilor din lumea reala
- clasa luminilor cu sursa si directie
- pozitia observatorului nu influenteaza culoarea obiectului in acest caz (intensitatea sa este aceeasi indiferent de pozitia observatorului)
- trebuie sa se determine cantitatea de lumina primita de obiect de la sursa de lumina: Legea lui Lambert
- Id = Ip * Kd * cos(i)
- Ip - intensitatea luminiii incidente
- Kd - este coeficientul de difuzie a luminii incidente, 0<=Kd<=1
- se mai inmulteste cu f_atenuare pentru diferentierea distantei de iluminare a obiectelor

Lumina speculara

- este componenta care da stralucirea obiectelor
- se aseamana cu lumina difuza, este focalizata
- lumina speculara nu poate fi fazuta din orice pozitie
- observatorul trebuie sa se afle chiar pe directia ei
- aceasta componenta depinde de pozitia observatorului
- Is = Ip * Ks * cos(q)^n
- Ks - coeficientul de reflexie speculara
- Ip - intensitatea luminii
- n - coeficient de reflexie speculara
- lumina speculara exista daca unghiul intre N si H e pana la 90 de grade

Calculul luminii

- Iluminarea de la o sursa de lumina i:
  - Iluminare_i = ambientala + difuza + speculara
- Daca sunt N surse de lumina:
  - Iluminarea totala pentru un punct P = suma(i..n Iluminare_i)

Modelul Lambert

- se vad trecerile de la o fata la alta
- Pentru fiecare fata se calculeaza o singura intensitate
  - I = k * (Normala * raza incidenta)
- N - este vectorul unitate normal la suprafata
- L - este versorul directiei sursei de lumina
- k - este constanta

Modelul Gouraud

- creste timpul de calcul fata de Lambert, insa vizibilitate mai buna
- se calculeaza o normala in fiecare varf al retelei poligonale
- se calculeaza o intensitate in fiecare varf al retelei
- se afiseaza suprafata calculand intensitatea intr-un punct al unui poligon
  - prin interpolarea liniara a intesitatilor varfurilor, pentru punctele de pe laturi
  - prin interpolare liniara intre intensitatile de pe laturi, pentru punctele interioare de pe fiecare lini de hasurare

Modelul Phong

- mai apropiat de culorile reale, dar mai greu de calculat
- Se calculeaza normala pentru fiecare varf al suprafetei
- Se calculeaza normala in fiecare punct interior suprafetei de afisat (prin interpolare)
  - prin interpolarea liniara a normalelor varfurilor, pentru punctele de pe laturi
  - prin interpolare liniara intre normalele de pe laturi, pentru punctele interioare de pe fiecare linie de hasurare
- Se calculeaza intensitatea in fiecare punct

> *Timpul de calcul de la mic la mare: Lambert (calcul pe fata) -> Gouraud (pe vertex) -> Phong (pe fragment)*

Ce e necesar pentru calculul luminii

- Atributele luminii (intensitatea luminii, culoare, pozitie, directie, forma)
- Atributele suprafetelor obiectelor (culoare, transparenta)
- Interactiunea dintre lumini si obiecte (orientarea obiectelor)
- Interactiunea dintre obiecte si observator (directia de vizualizare)

Calculul luminii in OpenGL

- Tipuri de lumini (pentru fiecare sursa de lumina):
  - Ambientala
  - Difuza
  - Speculara
- Lumina emisiva (lumina pe care un obiect o emite prin el insusi)
  - obiecte incandescente

Realizare iluminare

- Specificarea surselor de lumina
- Specificarea proprietatilor de material
- Specificarea normalelor

Specificare luminilor

- Lumina de tip spot
  - Directie spot D(x, y, z) in care este orientat spotul

- CUTOFF - deschiderea spotului
- EXPONENT - reprezinta concentratia luminii in interiorul spotului (poate fi constanta sau mai puternica spre centru si mai mica spre margini)
- spotEffect = (D . L)^spotExponent
- Componenta_speculara *= spotEffect

Iluminare in Vertex Shader

- Operatii realizate in vertex shader
  - Se transforma pozitia din spatiul obiect in viewport 3D
  - se calculeaza culoarea unui varf folosind componentele luminii: emisiva, ambientala, difuza, speculara

> *Calculul luminii se realizeaza in spatiul obiect*

Determinarea vizibilitatii

Z-buffer

- Algoritm in spatiul imagine
- Nu exista o ordine prestabilita pentru afisarea poligoanelor
- Pastreaza pixelul cel mai apropiat de ecran din cele care se suprapun dupa axa OZ
- Necesita ca structura de date un buffer avand un numar de elemente egal cu rezolutia ecranului
- Un element din buffer memoreaza informatia de adancime (z) asiciata pixelului (x, y)
- Pentru fiecare pixel este memorata si culoarea asociata
- Poate fi implementat hard si soft

- Initial adancimea e la infinit

Algoritmul de partitionare binara a spatiului

- Algoritmul in spatiul obiect
- Spatiul se partitioneaza prin plane de partitionare in mai multe zone
- Procesul de partitionare continua pana se ajunge la o zona cu un singur poligon
- In raport cu pozitia observatorului, algoritmul stabileste ordinea de afisare a seturilor de fete, pornind de la cea mai indepartata pana la cea mai apropiata
- Foloseste un arbore binar

Algoritmul Warnock

- Divizeaza recursiv zona de afisare, pana se poate determina poligonul vizibil in acea zona sau dimensiunea zonei este 1 pixel
- fata poligonala poate fi
  - acoperitor
  - intersectat
  - inclus
  - disjunct
- O zona de afisare cu dimensiunea mai mare de 1 pixel nu va mai fi divizata daca:
  - toate poligoanele sunt disjuncte fata de zona considerata
  - exista doar un poligon ce poate fi inclus sau acoperitor
  - exista mai multe poligoane incluse sau acoperitoare, dar cel mai apropiat de observator este un poligon acoperitor

Umbre

- Cand un observator priveste scena 3D iluminata de o sursa de lumina dintr-o pozitie diferita de aceea a sursei de lumina va vedea umbrele produse de obiectele scenei
- O zona de umbra se compune in realitate din doua parti:
  - umbra pura - este densa, neagra si are un contur bine delimitat
  - penumbra - inconjoara umbra pura. Zona aflata in penumbra primeste o parte din lumina de la sursa
- Sursele punctiforme produc numai umbra pura. Sursele reale, cu o dimensiune finita, produc si penumbra.
- Umbre proprii - sunt generate de obiectul insusi, care impiedica lumina sa ajunga la unele dintre fetele sale
- Umbre purtate - umbra pe care un obiect o produce pe alte parti ale scenei, la care lumina nu ajunge din cauza obiectului
- Umbre plane
- Harti de umbra (shadow map)
- Volume de umbra

Umbre plane

- Proiectia perspectiva pe un plan al umbrei cu centrul de proiectie in pozitia sursei de lumina
- Proiectia paralela pe un plan al umbrei, dupa directia sursei de lumina

Bufere in OpenGL

- Bufer de culoare
- Bufer de adancime
- Stencil bufer

Algoritmul z-buffer cu umbre

- Se porneste de la algoritmul z-buffer
- Redarea scenei din pozitia sursei de lumina cu testul de adancime activat -> shadow map avand coordonata de adancime stocata (corespunzatoare pozitiei sursei de lumina)
- rezoultia hartii de umbra trebuie sa fie aceeasi cu cea a bufferului de culoare. In caz contrar pot aparea artefacte.
- Algoritmul:
  - Redarea scenei din pozitia sursei de lumina
    - se stocheaza coordonata zs corespunzatoare fiecarui fragment
  - Redarea scenei din pozitia observatorului:
    - se transmite punctul corespunzator suprafetei in sistemul de coordonate corespunzator sursei de lumina -> (x, y, z)
    - se compara valoarea z cu valoarea zs(x, y) stocata la pasul 1
    - if zs mai aproape de sursa de lumina ca z
      - then (x, y) este umbrit
      - else (x, y) este luminat
- Creare shadow map
  - varf definit in spatiul obiect -> varf definit in spatiul viewportului
  - prin analogie spatiul viewport == spatiul de texturare -> fiecare varf se mapeaza pe un texel
- Accesare shadow map
  - Salvez zs - in textura -> transformare zs din intervalul (-1, 1) in intervalul (0, 1)

Volume de umbre

- Pentru fiecare poligon
  - piramida cu varful in sursa de lumina

    - poligonul de umbra - baza piramidei
- daca un punct P se afla in volumul de umbra al unei surse de lumina -> punctul nu va primi lumina de la acea sursa
- Raza R de la observator la fiecare punct:
    - incrementeaza contor cand raza intra in volumul de umbra (intersecteaza fete fata)
    - decrementeaza contor cand raza iesi din volumul de umbra (intersecteaza fete spate)
    - contor != 0 -> punctul este in umbra
- Daca intra in volum de umbra se incrementeaza
- Daca iese din volum de umbra se decrementeaza
- daca e 0 -> obiect neumbrit

Volume de umbra - z-pass

- folosire stencil buffer initializat cu 0
- poligoanele de umbra fata
    - daca test z-buffer adevarat:
        - incrementeaza contorul
- poligoanele de umbra spate
    - daca test z-buffer adevarat
        - decrementeaza contorul
- Algoritm:
    - afisare scena (fara iluminare). Se determina culorile astfel incat toate obiectele sunt in umbra. Bufferul de adancime este actualizat pentru toate punctele in scena
    - se dezactiveaza actualizarea bufferului de adancime si a bufferului de culoare
    - "afiseaza" fetele fata ale volumului de umbra. Se incrementeaza valoarea din stencil buffer pentru toate punctele pentru care testul z-buffer este

adevarat. Bufferul de adancime si bufferul de culoare sunt nemodificate
- "afiseaza" fetele spate ale volumului de umbra. Se decrementeaza valorile din stencil buffer pentru care testul-zbuffer este adevarat
- Activeaza folosirea bufferuli de adancime si de culoare
- Afiseaza scena cu iluminarea activata numai in cazul in care valoarea din stencil buffer este 0. Aceasta va suprascrie toate punctele anterior umbrite aflate in afara volumului de umbra.

Algoritmi de decupare

- Decuparea capetelor
  - x*min* <= *x* <= *x*max si y*min* <= *y* <= *y*min -> (x, y) punct interior
- Analiza capetelor unui segment de dreapta
  - acceptare
  - rejectare
  - intersectie
- Rezolvarea ecuatiilor folosind y = mx + b - ecuatia dreptei

Algoritm de decupare linii Cohen-Sutherland

- Decuparea se realizeaza fata de o zona dreptunghiulara
  - segment inclus - ambele capete in interior
  - segment disjunct - ambele capete in exterior
  - segment intersectat - trebuie sa se calculeze punctele de intersectie

- Se considera punctele max/min
  - x*min* <= *x* <= *x*max si y*min* <= *y* <= *y*min -> (x, y) punct interior

- Se foloseste codificare

    - b4 = 1 y > y_max
    - b3 = 1 y < y_min
    - b2 = 1 x > x_max
    - b1 = 1 x < x_min
- Se verifica acceptare si rejectare

Decuparea poligoanelor cu algoritmul Cohen-Sutherland

- s - varful poligonului analizat la pasul anterior
- p - varful poligonului analizat la pasul curent
- se pastreaza doar varful din zona de interior

Decupare in 3D cu algoritmul Cohen-Sutherland

- Zona de decupare delimitata de planele corespunzatoare volumului de vizualizare

Iluminare Globala

Ray Casting

- Genereaza raze
- Gaseste cel mai apropiat punct de intersectie
- Rezulta culoarea

- Algoritm
    - pentru fiecare fragment
        - se construieste raza de observator prin zona de afisare
        - se determina prima suprafata intersectata de raza
        - se calculeaza culoarea pixelului bazata pe culoarea suprafetei obtinuta la intersectie

Ray-Tracing

- Simuleaza razele de lumina de la sursa de lumina la observator
- Razele pornesc de la sursa de lumina
- Cost mare
- Ray-Tracing ("Backward")
    - razele pornesc de la observator

Clasificarea razelor:

- Raza observator
- Normala
- Raza reflectata
- Raza refractata
- Raza de lumina / umbra

Algoritmul Ray-Tracing

- pentru fiecare pixel
    - genereaza raza corespunzatoare observatorului si zonei de afisare
    - obtine culoarea
        - daca adancimea e mai mare ca adancimea maxima ->

        - culoarea neagra
    - altfel
        - determina punctul de intersectie cel mai apropiat
        - calculeaza in mod recursiv propagari succesive
  - scrie pixel

Structuri de accekerare a determinarii intersectiilor

- Volume incadratoare
- Ierarhii de volume incadratoare
- Se verifica intersectia cu volumul incadrator
  - reduce numarul de intersectii si complexitatea lor
- Suprafete subtiri - poate fi ignorata modificarea directiei

Algoritmi de generare segmente de dreapta

- se determina punctele de pe dreapta
- se aleg pixelii care aproximeaza cel mai bine segmentul teoretic

Algoritmul Bresenham pentru segmente de dreapta

- Segmente de dreapta cu panta 0 <= m <= 1
- crestem x si calculam y
- se alege pixelul cel mai apropiat
- Pasi
  - dx = x2 - x1; dy = y2 - y1;

    - c1 = 2 * dy;
    - c2 = c1- 2 * dx;
    - t = c1 - dx
- se porneste de la pixelul curent
- se va face alegerea intre A si B pe baza calculului de eroare (d1- d2) * dx - care e mai apropiat
- se calculeaza care pixel trebuie colorat

Algoritmul Bresenham pentru segmente de dreapta - generalizare

Alegerea urmatorului punct de aproximare se face prin: - Deplasament orizontal / vertical - Deplasament diagonal

Algoritmul Bresenham pentru generarea cercurilor

- Pe baza (x, y) generez alte 7 puncte pe baza simetriei
- Se genereaza punctele de pe cerc pentru x, y > 0 si x < y
- Restul punctelor se genereaza prin simetrie

Generare suprafetelor definite prin contur

- Algoritmi bazati pe verificarea culorii punctelor vecine punctului curent
- Algoritmi bazati pe parcurgerea liniilor raster care traverseaza suprafata
    - Latura L, linia de hasurare yh: y*min*L <= yh < y*max*L

Algoritm

- for yh la ymax*polig - d pana la ymin*polig cu pas = -d
    - se calculeaza intersectiile dreptei y = yh cu laturile poligonului
    - se ordoneaza punctele de intersectie crescator dupa abscisa
    - se afiseaza segmente de dreapta intre primul si al doilea punct de intersectie, intre al treilea si al patrulea punct de intersectie

Published with Simplenote